Báo cáo quá trình học việc

Nền tảng hạ tầng: Linux doanh nghiệp, Ảo hóa và Container

Người trình bày: Lê Tấn Lực (Intern) | Tháng 11/2026

Mở đầu

Trong thời gian học việc vừa qua, em tập trung học kiến thức nền tảng về hạ tầng hệ thống, gồm ba mảng chính: hệ điều hành Linux dòng doanh nghiệp, ảo hóa và container. Em thực hành trên các máy ảo chạy **Rocky Linux** (bản tương thích với Red Hat Enterprise Linux) và dùng **Docker** để chạy container. Riêng phần hypervisor và vòng đời VM, em mới tìm hiểu ở mức lý thuyết.

Thay vì học từng chủ đề rời rạc, em sắp xếp kiến thức theo đúng thứ tự mà một máy chủ cần có để đi vào hoạt động. Trước hết phải có máy, và trong doanh nghiệp đó thường là máy ảo chạy trên hypervisor. Máy cần được nối mạng để giao tiếp với các máy khác, cần ổ đĩa để lưu dữ liệu, cần tài khoản người dùng với quyền hạn phù hợp, cần chạy dịch vụ và được firewall bảo vệ. Khi hệ thống đã chạy thì phải theo dõi được tài nguyên CPU, RAM, disk. Cuối cùng là container, cách đóng gói và vận hành ứng dụng phổ biến hiện nay. Sắp xếp như vậy giúp em thấy phần trước luôn là tiền đề cho phần sau.

Báo cáo đi theo đúng thứ tự đó. Phần ảo hóa em trình bày lý thuyết. Các phần còn lại gồm những gì em hiểu, ví dụ lệnh em đã chạy khi thực hành, và các lỗi hoặc bài học em rút ra.

1. Hypervisor và vòng đời VM (phần lý thuyết)

Ở mảng này em mới dừng lại ở mức tìm hiểu lý thuyết, chưa thực hành bài bản. Em vẫn trình bày trước vì đây là "nền móng" của mọi phần phía sau: trước khi có một máy chủ Linux để cấu hình, ta cần có máy, và trong doanh nghiệp, máy đó hầu hết là máy ảo.

Hypervisor là gì

Hypervisor là lớp phần mềm chia tài nguyên của một máy vật lý (CPU, RAM, ổ đĩa, card mạng) thành nhiều máy ảo (VM) độc lập. Mỗi VM tưởng rằng mình đang chạy trên một máy thật riêng: có CPU ảo (vCPU), RAM, ổ đĩa ảo và card mạng ảo. Thực tế, hypervisor đứng giữa điều phối để các VM dùng chung phần cứng mà không ảnh hưởng lẫn nhau. Các CPU hiện đại có sẵn tính năng hỗ trợ ảo hóa (Intel VT-x, AMD-V) để việc này đạt hiệu năng gần như máy thật.

Em lấy ví dụ: một máy chủ vật lý có 32 CPU và 128 GB RAM, nếu chỉ chạy một ứng dụng thì phần lớn thời gian tài nguyên bị bỏ phí. Khi ảo hóa, cùng máy đó có thể chạy 10 đến 15 VM cho web, database, mail... Doanh nghiệp tiết kiệm phần cứng, điện, chỗ đặt máy, và tạo máy mới chỉ mất vài phút thay vì chờ mua thiết bị.

Hai loại hypervisor

Type 1 (bare-metal) chạy trực tiếp trên phần cứng, không cần hệ điều hành bên dưới. Ví dụ: VMware ESXi, Microsoft Hyper-V, KVM, Proxmox. Loại này dùng trong datacenter và cloud vì hiệu năng cao, ổn định. **Type 2 (hosted)** chạy như một ứng dụng trên hệ điều hành có sẵn, ví dụ VirtualBox, VMware Workstation. Loại này phù hợp cho cá nhân học tập, thử nghiệm.

Một điểm em thấy thú vị: Hyper-V trông giống một tính năng của Windows nhưng lại là Type 1. Khi bật Hyper-V, hypervisor được nạp xuống dưới cùng, còn chính Windows trở thành một phân vùng

chạy trên nó. Đây cũng là lý do khi bật Hyper-V, VirtualBox trên cùng máy có thể chạy chậm đi, vì hai hypervisor phải dùng chung tính năng ảo hóa của CPU. Các dịch vụ cloud cũng dựa trên nguyên lý này, ví dụ máy ảo EC2 của AWS chạy trên hypervisor Nitro (dựa trên KVM).

Một VM gồm những gì

Về bản chất, một VM chỉ là một nhóm file: một file cấu hình mô tả phần cứng ảo (số vCPU, dung lượng RAM, card mạng nối vào đâu) và một hoặc nhiều file ổ đĩa ảo chứa toàn bộ hệ điều hành và dữ liệu. Mỗi hypervisor có định dạng ổ đĩa riêng: VDI của VirtualBox, VHDX của Hyper-V, VMDK của VMware, QCOW2 của KVM. Ổ đĩa ảo có hai kiểu cấp phát: **cấp phát động** (thin, file lớn dần theo dữ liệu thật, tiết kiệm chỗ) và **cấp phát cố định** (thick, chiếm đủ dung lượng ngay từ đầu, hiệu năng ổn định hơn). Vì VM chỉ là file nên có thể sao chép, di chuyển hay sao lưu nó dễ dàng như sao chép dữ liệu.

Em cũng tìm hiểu khái niệm **overcommit**: hypervisor cho phép cấp tổng số vCPU hoặc RAM cho các VM vượt quá tài nguyên thật, vì không phải VM nào cũng dùng hết cùng lúc. Overcommit giúp tận dụng tài nguyên, nhưng nếu quá tay thì khi các VM cùng bận, tất cả sẽ chậm. Đây là bài toán cân bằng giống phần quản lý tài nguyên em trình bày ở phần 6.

Vòng đời của một VM

Một VM thường đi qua các giai đoạn sau. **Tạo**: chọn loại hệ điều hành, cấp vCPU, RAM, ổ đĩa, card mạng. **Cài đặt và cấu hình**: cài hệ điều hành, đặt IP, tạo user, cài dịch vụ (chính là nội dung các phần 2 đến 5). **Vận hành**: theo dõi tài nguyên, điều chỉnh khi thiếu, ví dụ tăng RAM hoặc thêm ổ. **Bảo trì**: snapshot trước khi cập nhật, backup định kỳ, clone ra máy mới. **Thu hồi**: khi hết sử dụng thì backup lần cuối nếu cần rồi xóa, để giải phóng tài nguyên cho VM khác.

Trong doanh nghiệp, người ta thường không cài từng VM từ đầu mà tạo một **template** (máy mẫu đã cài sẵn hệ điều hành và cấu hình chuẩn), rồi **clone** ra hàng loạt máy giống nhau. Cách này nhanh và đảm bảo các máy đồng nhất. Hypervisor Type 1 còn hỗ trợ **live migration**, tức chuyển một VM đang chạy sang máy vật lý khác mà không phải tắt, rất hữu ích khi cần bảo trì phần cứng.

Snapshot khác backup ở điểm nào

Đây là điểm em thấy quan trọng nhất trong phần này. **Snapshot** lưu lại trạng thái VM tại một thời điểm. Sau khi chụp, ổ đĩa gốc được "đóng băng", mọi thay đổi mới ghi vào một file chênh lệch riêng. Nhờ vậy, muốn quay về chỉ cần bỏ file chênh lệch, gần như tức thì. Nhưng chính cơ chế này cho thấy snapshot **phụ thuộc vào ổ đĩa gốc** và nằm ngay cạnh VM. Nếu ổ cứng máy chủ hỏng thì mất cả VM lẫn snapshot.

Backup là một bản sao đầy đủ, độc lập, lưu ở nơi khác, nên khôi phục được kể cả khi mất toàn bộ máy chủ. Em ví snapshot như nút "Undo" trong Word, còn backup như bản sao tài liệu cất ở két sắt nhà khác. Nguyên tắc em tìm hiểu được là **3-2-1**: giữ 3 bản dữ liệu, trên 2 loại thiết bị lưu trữ khác nhau, trong đó 1 bản đặt ở nơi khác.

Ví dụ tình huống: trước khi cập nhật hệ điều hành cho máy chủ web, quản trị viên chụp snapshot. Nếu bản cập nhật gây lỗi, chỉ mất vài giây để quay lại. Cập nhật thành công thì nên xóa snapshot đi, vì giữ lâu khiến file chênh lệch phình to và VM chạy chậm dần. Còn backup thì vẫn chạy hằng đêm, độc lập với snapshot.

Hiểu được cách một VM ra đời và được bảo vệ, câu hỏi tiếp theo là làm sao để các máy kết nối được với nhau và ra Internet.

2. Network: IP, routing, DNS và kết nối giữa các VM

Mỗi máy trong mạng cần một địa chỉ IP và biết mình thuộc mạng nào thông qua subnet mask. Ví dụ **192.168.56.10/24** nghĩa là máy nằm trong mạng 192.168.56.0, mạng này có tối đa 254 địa chỉ dùng được. Khi gửi gói tin đến một máy cùng mạng, máy gửi thẳng đến đó. Khi đích nằm ở mạng khác, gói tin phải đi qua **gateway** (router). Quyết định "đi đường nào" chính là **routing**.

Các chế độ mạng của VM (lý thuyết) và cấu hình IP trong Linux

Phần này em mới tìm hiểu ở mức lý thuyết. Hypervisor cho VM nhiều chế độ mạng, mỗi chế độ trả lời một câu hỏi khác nhau. **NAT**: VM ra được Internet nhưng bên ngoài không truy cập vào được. **Host-only**: máy thật và các VM thấy nhau nhưng không ra Internet. **Bridged**: VM giống như một máy thật trong mạng LAN, nhận IP cùng dải với các máy khác. **Internal**: chỉ các VM nói chuyện với nhau. Trên Hyper-V, các khái niệm tương ứng là External, Internal và Private switch. Một cách bố trí phổ biến là cho mỗi VM hai card mạng: một card NAT để tải phần mềm, một card mạng nội bộ để quản trị và để các máy kết nối với nhau.

Phần em đã thực hành là cấu hình mạng **bên trong Linux**. Trên máy Rocky Linux có sẵn hai card mạng, em kiểm tra trạng thái card và đặt IP tĩnh cho card thứ hai bằng nmcli:

```
nmcli device status
nmcli con mod enp0s8 ipv4.method manual ipv4.addresses 192.168.56.10/24
nmcli con up enp0s8
ip addr show enp0s8
```

Đọc bảng định tuyến

Lệnh ip route cho em biết máy sẽ gửi gói tin đi đâu:

```
$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.10
```

Dòng đầu tiên là đường mặc định: mọi gói tin ra Internet đi qua gateway 10.0.2.2 của card NAT. Dòng cuối cho biết gói tin đến các VM khác trong mạng 192.168.56.0/24 đi thẳng qua enp0s8.

> *Bài học: Lúc đầu em khai báo gateway cho cả hai card, nên máy có hai đường mặc định và bị mất Internet. Từ đó em hiểu mỗi máy chỉ nên có một default route, trừ khi có chủ đích định tuyến phức tạp hơn.*

DNS và tên máy

Con người nhớ tên, máy tính dùng IP, nên DNS làm nhiệm vụ chuyển tên thành IP. Em cấu hình DNS server qua NetworkManager (để không bị ghi đè file /etc/resolv.conf). Trong lab nhỏ, em khai báo tên các máy trong /etc/hosts để gọi nhau bằng tên:

```
nmcli con mod enp0s3 ipv4.dns "8.8.8.8 1.1.1.1"
echo "192.168.56.10  web01" >> /etc/hosts
echo "192.168.56.11  db01"  >> /etc/hosts
ping -c 3 db01
dig google.com +short        # gói bind-utils
```

Quan trọng nhất ở phần này là em có được **thứ tự kiểm tra khi mất kết nối**: card mạng có IP chưa (ip addr), có đường đi chưa (ip route), ping được gateway không, ping được IP đích không, rồi mới đến ping bằng tên (DNS). Kiểm tra từng lớp như vậy giúp khoanh vùng lỗi nhanh hơn nhiều so với đoán mò. Khi mạng đã thông, em chuyển sang chuẩn bị chỗ lưu dữ liệu cho máy chủ.

3. Storage: partition, LVM, filesystem, mount

Một nguyên tắc em học được là không nên để dữ liệu ứng dụng chung ổ với hệ điều hành. Nếu log hoặc dữ liệu web làm đầy ổ, cả máy chủ có thể ngừng hoạt động. Vì vậy em gắn thêm một ổ ảo 10 GB vào VM web01 trên VirtualBox và đi qua đủ các bước: nhận ổ, chia partition, gom vào LVM, tạo filesystem và mount vào thư mục /data.

Em hình dung **LVM** theo ba lớp. **Physical Volume (PV)** là các ổ đĩa hoặc partition thật. **Volume Group (VG)** giống một "bể chứa" gom dung lượng của nhiều PV lại. **Logical Volume (LV)** là các phần cắt ra từ bể đó để sử dụng. Lợi ích lớn nhất là khi hết dung lượng, chỉ cần thêm ổ vào bể rồi nới rộng LV, không phải chia lại ổ từ đầu.

```
lsblk                                  # ổ mới xuất hiện là /dev/sdb
fdisk /dev/sdb                         # tạo partition sdb1, đổi type sang Linux LVM
pvcreate /dev/sdb1
vgcreate vg_data /dev/sdb1
lvcreate -n lv_web -L 5G vg_data
mkfs.xfs /dev/vg_data/lv_web           # XFS là filesystem mặc định của RHEL
mkdir /data
mount /dev/vg_data/lv_web /data
df -h /data
```

Lệnh mount chỉ có hiệu lực đến lần khởi động lại. Để mount tự động, em khai báo trong /etc/fstab và dùng UUID thay cho tên thiết bị, vì tên kiểu /dev/sdb có thể thay đổi khi thêm bớt ổ:

```
blkid /dev/vg_data/lv_web
# Thêm dòng sau vào /etc/fstab
UUID=3f1c...e9a2   /data   xfs   defaults   0 0
mount -a          # kiểm tra fstab TRƯỚC khi reboot
```

> *Bài học: Nếu một dòng trong fstab bị sai, máy sẽ rơi vào emergency mode ở lần khởi động sau. Vì vậy em luôn chạy mount -a để kiểm tra fstab trước khi reboot.*

Khi /data sắp đầy, em nới rộng LV ngay lúc máy đang chạy. Tùy chọn -r giúp nới rộng luôn cả filesystem. Lưu ý là XFS chỉ mở rộng được, không thu nhỏ được, nên ngay từ đầu cần cấp dung lượng vừa phải và nới dần khi cần.

```
lvextend -r -L +3G /dev/vg_data/lv_web
vgextend vg_data /dev/sdc1             # khi VG hết chỗ: thêm ổ mới vào bể
```

Có chỗ chứa dữ liệu rồi, câu hỏi tiếp theo là ai được phép đọc, ghi vào đó.

4. User, group và permission

Linux là hệ điều hành nhiều người dùng, nên mọi file đều trả lời ba câu hỏi: ai sở hữu (owner), thuộc nhóm nào (group), và những người còn lại (other) được làm gì. Mỗi đối tượng có ba quyền: **r** (đọc, giá trị 4), **w** (ghi, giá trị 2), **x** (thực thi, hoặc vào được thư mục, giá trị 1). Cộng các giá trị lại sẽ ra con số quen thuộc, ví dụ 750 nghĩa là owner rwx, group r-x, other không có quyền gì.

Em thực hành với một tình huống cụ thể: team web gồm hai bạn An và Bình cùng làm việc trong thư mục /data/web, còn người ngoài nhóm không được xem.

```
groupadd webteam
useradd -m -G webteam an
useradd -m -G webteam binh
passwd an
mkdir /data/web
```

```
chown root:webteam /data/web
chmod 2770 /data/web
ls -ld /data/web
drwxrws---. 2 root webteam 6 Nov  2 10:00 /data/web
```

Con số 2 ở đầu là **setgid**, thể hiện bằng chữ s trong rws. Nhờ nó, mọi file mới tạo trong thư mục đều tự thuộc nhóm webteam. Nếu không có setgid, file của An sẽ thuộc nhóm riêng của An và Bình không sửa được. Thông tin người dùng và nhóm được lưu trong /etc/passwd, /etc/group, còn mật khẩu đã mã hóa nằm trong /etc/shadow.

Quyền quản trị và nguyên tắc least privilege

Trên RHEL, thành viên nhóm **wheel** được dùng sudo với toàn quyền. Tuy nhiên em học được nguyên tắc **least privilege**: chỉ cấp đúng quyền cần thiết. Ví dụ team web chỉ cần khởi động lại dịch vụ web, nên em cấp đúng một lệnh đó:

```
visudo -f /etc/sudoers.d/webteam
%webteam ALL=(root) /usr/bin/systemctl restart httpd
```

> *Bài học: Lệnh usermod -G wheel an (thiếu -a) sẽ ghi đè danh sách nhóm và xóa An khỏi webteam. Phải dùng usermod -aG để thêm nhóm mới mà vẫn giữ các nhóm cũ.*

Lệnh sudo ở trên nhắc đến httpd, đó chính là dịch vụ web em sẽ dựng ở phần tiếp theo.

5. Service (systemd) và firewall

Trên RHEL, **systemd** là tiến trình đầu tiên chạy khi máy khởi động (PID 1), có nhiệm vụ quản lý mọi dịch vụ. Mỗi dịch vụ được mô tả bằng một **unit file**. Em phân biệt hai khái niệm dễ nhầm: **start** là chạy dịch vụ ngay bây giờ, còn **enable** là cho dịch vụ tự chạy mỗi khi khởi động máy. Muốn làm cả hai cùng lúc thì dùng enable --now.

```
dnf install -y httpd
systemctl enable --now httpd
systemctl status httpd
journalctl -u httpd -n 50 --no-pager     # xem 50 dòng log gần nhất của dịch vụ
```

Để hiểu sâu hơn, em tự viết một unit file cho ứng dụng nhỏ, chạy dưới quyền user An (không chạy bằng root, đúng tinh thần phần 4):

```
# /etc/systemd/system/hello-app.service
[Unit]
Description=Hello App demo
After=network.target

[Service]
User=an
ExecStart=/usr/bin/python3 -m http.server 8080 --directory /data/web
Restart=on-failure

[Install]
WantedBy=multi-user.target

systemctl daemon-reload
systemctl enable --now hello-app
kill -9 $(systemctl show -p MainPID --value hello-app)   # thử "giết" tiến trình
systemctl status hello-app                               # systemd đã tự chạy lại
```

Thử nghiệm này cho em thấy rõ giá trị của Restart=on-failure: dịch vụ bị lỗi sẽ được systemd tự khởi động lại mà không cần người can thiệp.

Firewall với firewalld

Dịch vụ chạy rồi nhưng từ Windows vẫn chưa truy cập được, vì **firewalld** mặc định chặn các cổng chưa được mở. firewalld tổ chức theo **zone**, mỗi card mạng thuộc một zone, và trong mỗi zone ta cho phép service hoặc port cụ thể.

```
firewall-cmd --get-active-zones
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --reload
firewall-cmd --list-all
curl http://192.168.56.10          # chạy từ máy Windows để kiểm tra
```

> *Bài học: Thêm rule mà không có --permanent thì rule sẽ mất sau khi reload hoặc reboot. Ngược lại, có --permanent mà quên reload thì rule chưa có hiệu lực.*

Khi em đổi thư mục web của httpd sang /data/web, trình duyệt báo lỗi 403 dù quyền rwx đã đúng. Em tìm ra nguyên nhân là **SELinux**, lớp bảo mật mặc định của RHEL, chặn httpd đọc thư mục không có nhãn phù hợp. Em mới tìm hiểu ở mức cơ bản, nhưng đã biết cách gán lại nhãn:

```
semanage fcontext -a -t httpd_sys_content_t "/data/web(/.*)?"
restorecon -Rv /data/web
```

Từ lỗi này em rút ra: "web không vào được" có thể do dịch vụ chết, firewall chặn, sai quyền, hoặc SELinux chặn. Phải kiểm tra lần lượt từng lớp, giống cách kiểm tra mạng ở phần 2. Khi hệ thống đã chạy ổn, việc tiếp theo là theo dõi xem máy chủ có "khỏe" không.

6. Linux doanh nghiệp: quản lý tài nguyên máy chủ

Em tự đặt ra tình huống "người dùng báo web01 chạy chậm" và tạo tải giả lập: lệnh yes > /dev/null để chiếm CPU, lệnh dd ghi file lớn để làm đầy disk. Sau đó em tập điều tra theo thứ tự CPU, RAM, disk rồi đến tiến trình.

CPU và load average

```
nproc                    # máy có 2 CPU
uptime                   # load average: 3.95, 2.10, 1.02
top                      # xem %CPU, %MEM và chỉ số wa (I/O wait)
```

Load average 3.95 trên máy 2 CPU nghĩa là lượng công việc đang chờ nhiều gần gấp đôi khả năng xử lý, tức máy đang quá tải. Trong top, nếu chỉ số **wa** cao thì nghẽn nằm ở disk chứ không phải CPU. Chi tiết này giúp em không kết luận vội.

RAM

```
free -h
               total   used   free   shared  buff/cache  available
Mem:           1.9Gi   620Mi  110Mi   12Mi     1.2Gi       1.1Gi
```

Lúc đầu em lo vì cột free chỉ còn 110 MB. Sau đó em hiểu Linux tận dụng RAM trống làm cache để tăng tốc, và con số cần nhìn là cột **available** (1.1 GB). Dấu hiệu thiếu RAM thật là swap tăng liên tục, hoặc log có dòng "Out of memory" (xem bằng dmesg | grep -i oom).

Disk và tiến trình

```
df -h                                  # phân vùng nào đầy?
```

```
du -sh /var/log/* | sort -h | tail -5    # thư mục nào chiếm nhiều nhất?
ps aux --sort=-%cpu | head -5            # tiến trình nào ngốn CPU nhất?
kill -15 2345                            # yêu cầu dừng nhẹ nhàng trước
kill -9 2345                             # chỉ dùng khi tiến trình không phản hồi
renice -n 10 -p 3456                     # hạ ưu tiên cho tiến trình backup
```

Phát hiện quan trọng nhất của em ở phần này là systemd có thể giới hạn tài nguyên cho từng dịch vụ:

```
systemctl set-property hello-app.service CPUQuota=50% MemoryMax=256M
```

Cơ chế đứng sau lệnh này là **cgroups** của Linux kernel. Đây cũng chính là cơ chế Docker dùng để giới hạn tài nguyên cho container. Phát hiện này dẫn em sang phần cuối cùng.

7. Container: image, registry, volume, network, resource limit

Sau khi làm việc với VM, em thấy rõ sự khác biệt của container. **VM ảo hóa phần cứng**: mỗi VM có kernel và hệ điều hành riêng, nặng vài GB và khởi động mất vài phút. **Container ảo hóa ở mức hệ điều hành**: các container dùng chung kernel của máy chủ, được cô lập bằng **namespaces** (mỗi container thấy tiến trình, mạng, file riêng) và giới hạn tài nguyên bằng **cgroups** (cơ chế em đã gặp ở phần 6). Vì vậy container nhẹ hơn nhiều và khởi động chỉ trong vài giây. Em hình dung VM giống những căn nhà riêng, mỗi nhà có móng riêng, còn container giống các căn hộ chung cư dùng chung móng nhưng mỗi căn có cửa khóa riêng.

Em cài Docker trên web01. Docker cũng chỉ là một dịch vụ systemd (phần 5). Ngoài ra em lưu ý rằng thành viên nhóm docker có quyền gần như root, nên chỉ thêm người thật sự cần (phần 4).

```
dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
systemctl enable --now docker
```

Image và registry

Image là bản đóng gói chỉ đọc gồm ứng dụng và mọi thứ ứng dụng cần để chạy. Image được xây theo từng lớp (layer), và Docker cache lại các lớp không đổi nên build lần sau rất nhanh. Em đóng gói trang web tĩnh của mình như sau:

```
# Dockerfile
FROM nginx:1.27-alpine
COPY site/ /usr/share/nginx/html/
EXPOSE 80

docker build -t mysite:1.0 .
docker history mysite:1.0          # xem các layer của image
```

Registry là nơi lưu trữ và phân phối image, giống "kho ứng dụng" cho container. Ngoài Docker Hub, em tự dựng một registry nội bộ ngay trong lab để web01 đẩy image lên và db01 kéo về. Em cũng tập thói quen gắn tag phiên bản cụ thể như 1.0 thay vì latest, để biết chính xác phiên bản nào đang chạy.

```
docker run -d -p 5000:5000 --name registry registry:2
docker tag mysite:1.0 192.168.56.10:5000/mysite:1.0
docker push 192.168.56.10:5000/mysite:1.0
docker pull 192.168.56.10:5000/mysite:1.0     # chạy trên db01
```

Lưu ý: registry nội bộ trong lab chạy HTTP không mã hóa, nên phải khai báo "insecure-registries" trong /etc/docker/daemon.json trên các máy kéo image. Môi trường thật thì

cần dùng HTTPS.

Volume: dữ liệu không mất khi container bị xóa

Container được thiết kế để có thể xóa đi, tạo lại bất cứ lúc nào, và dữ liệu ghi bên trong container cũng mất theo. Vì vậy dữ liệu cần giữ lâu dài (như database) phải đặt ra **volume**. Em kiểm chứng bằng cách xóa container PostgreSQL rồi tạo lại với cùng volume, và dữ liệu vẫn còn nguyên:

```
docker volume create pgdata
docker run -d --name db -e POSTGRES_PASSWORD=secret \
    -v pgdata:/var/lib/postgresql/data postgres:16
docker rm -f db          # xóa container, chạy lại lệnh trên: dữ liệu vẫn còn
```

Em cũng dùng **bind mount** để gắn thư mục /data/web (phần 3) vào container nginx. Trên RHEL cần thêm hậu tố :Z để Docker gán nhãn SELinux phù hợp. Đây lại là lỗi 403 em từng gặp ở phần 5:

```
docker run -d --name web -p 8081:80 -v /data/web:/usr/share/nginx/html:ro,Z nginx:1.27-alpine
```

Network giữa các container

Mạng bridge mặc định của Docker không phân giải được tên container. Khi em tạo một **user-defined network**, Docker có sẵn DNS nội bộ, nên container web gọi database chỉ cần dùng tên "db", không cần nhớ IP. Đây là ý tưởng DNS ở phần 2, chỉ ở quy mô nhỏ hơn.

```
docker network create appnet
docker run -d --name db  --network appnet -v pgdata:/var/lib/postgresql/data \
    -e POSTGRES_PASSWORD=secret postgres:16
docker run -d --name web --network appnet -p 8080:80 mysite:1.0
```

> *Bài học: khi publish port (-p), Docker tự thêm rule firewall riêng. Vì vậy port của container có thể mở ra ngoài dù em chưa khai báo trong firewalld. Cần cẩn thận, chỉ publish những port thật sự cần.*

Giới hạn tài nguyên

Nếu không giới hạn, một container lỗi có thể chiếm hết RAM và làm ảnh hưởng mọi container khác. Em giới hạn CPU và RAM rồi theo dõi bằng docker stats. Khi container vượt quá mức RAM cho phép, kernel sẽ "kill" nó (OOMKilled), đúng như cơ chế cgroups em đã thấy với systemd:

```
docker run -d --name web --memory 256m --cpus 0.5 mysite:1.0
docker stats --no-stream
docker inspect web --format '{{.State.OOMKilled}}'
```

Tổng kết: một bài lab nối tất cả lại với nhau

Để kiểm tra mình có thật sự hiểu hay không, em làm lại toàn bộ từ đầu thành một bài lab xuyên suốt: **dựng hệ thống web hai tầng trên hai VM**. Quá trình như sau.

Trên hai máy ảo web01 và db01 chạy Rocky Linux, em đặt IP tĩnh trong dải 192.168.56.0/24 bằng nmcli, và khai báo tên trong /etc/hosts để hai máy gọi nhau bằng tên. Trên web01, em gắn thêm ổ, tạo LVM và mount vào /data. Sau đó em tạo nhóm webteam và phân quyền 2770 cho /data/web. Dịch vụ httpd được bật bằng systemd, firewalld chỉ mở HTTP và SSH, SELinux được gán đúng nhãn. Em giả lập tải để luyện kỹ năng theo dõi bằng top, free, df. Cuối cùng em chuyển ứng dụng sang container: image lưu ở registry nội bộ, database dùng volume, các container nói chuyện qua user-defined network, và mỗi container đều có giới hạn RAM, CPU.

Qua quá trình này, em rút ra bốn điều:

Thứ nhất, các mảng kiến thức không tách rời nhau. Một lỗi đơn giản như "không vào được web" có thể nằm ở mạng, firewall, quyền file, SELinux hoặc dịch vụ đã chết. Người làm hạ tầng cần nhìn được toàn bộ các lớp đó.

Thứ hai, cần kiểm tra theo từng lớp, từ dưới lên trên, thay vì đoán mò.

Thứ ba, đọc log trước khi sửa. Các lệnh journalctl, systemctl status và dmesg thường đã chỉ ra nguyên nhân.

Thứ tư, luôn có đường lui. Phải snapshot hoặc backup trước mọi thay đổi lớn.

Em cũng tự đánh giá thẳng thắn: em đã nắm được thao tác cơ bản và hiểu "vì sao" phía sau từng thao tác. Tuy nhiên vẫn còn nhiều phần em mới ở mức làm quen hoặc lý thuyết: vòng đời VM (snapshot, backup, template), SELinux, chia subnet phức tạp, xử lý sự cố hiệu năng trong môi trường thật, và viết script tự động hóa. Đây cũng là lý do định hướng sắp tới bắt đầu bằng việc củng cố.

Định hướng sắp tới

Lộ trình của em đi theo một logic nối tiếp. Đầu tiên là làm chắc nền tảng hiện tại. Sau đó học Kubernetes, bước tự nhiên tiếp theo sau container. Rồi đưa toàn bộ kiến thức lên cloud với chứng chỉ AWS. Song song với đó là đào sâu Linux doanh nghiệp theo chuẩn Red Hat.

Giai đoạn 1: Củng cố nền tảng (thực hiện liên tục)

Việc đầu tiên là biến phần lý thuyết ảo hóa thành thực hành: tự tạo VM bằng dòng lệnh trên VirtualBox và Hyper-V, chụp snapshot trước mỗi thay đổi lớn, export VM ra file để backup và thử khôi phục, tạo template rồi clone ra nhiều máy. Song song, em sẽ làm lại các bài lab Linux mà không xem hướng dẫn, cho đến khi tự tin xử lý được sự cố. Em sẽ viết các script Bash để tự động hóa những việc lặp lại, như tạo user hàng loạt, backup thư mục định kỳ bằng systemd timer, và kiểm tra tình trạng máy chủ. Em cũng sẽ ghi chép lại các lỗi đã gặp cùng cách xử lý, thành tài liệu nội bộ để tra cứu.

Giai đoạn 2: Kubernetes (K8s)

Docker giải quyết bài toán chạy container trên một máy. Kubernetes giải quyết bài toán điều phối hàng trăm container trên nhiều máy: tự khởi động lại container khi lỗi, tự mở rộng khi tải tăng, cập nhật phiên bản không gián đoạn. Điều em thấy thú vị là hầu hết khái niệm của K8s đều nối với những gì em vừa học. **Service** tương ứng với network và DNS. **PersistentVolume** tương ứng với volume và storage. **Requests và limits** chính là cgroups. **kubelet** trên mỗi node là một dịch vụ systemd. Em dự định dựng cụm K8s bằng kubeadm trên ba VM Rocky Linux (một control plane, hai worker) đây cũng là dịp áp dụng phần vòng đời VM (tạo từ template, snapshot trước khi nâng cấp cụm) cùng kỹ năng cấu hình mạng.

Giai đoạn 3: Chứng chỉ AWS Solutions Architect – Associate (SAA-C03)

Kỳ thi SAA-C03 đánh giá khả năng thiết kế giải pháp trên AWS theo Well-Architected Framework, gồm bốn nhóm: bảo mật (30%), khả năng phục hồi (26%), hiệu năng (24%) và tối ưu chi phí (20%). Em chọn chứng chỉ này vì kiến thức nền vừa học ánh xạ gần như trực tiếp sang các dịch vụ AWS. **EC2** là VM. **VPC, subnet, route table** là network và routing. **Security Group** là firewall. **EBS, EFS, S3** là storage. **IAM** là user, group và least privilege. **EBS snapshot và AWS Backup** là snapshot và backup. **ECR, ECS, EKS** là registry, container và Kubernetes. **CloudWatch** là theo dõi tài nguyên. Nhờ vậy, em không phải học AWS từ con số không mà chỉ cần học cách các khái

niệm quen thuộc vận hành trên cloud. Em sẽ kết hợp học lý thuyết với làm lab thực tế trên AWS Free Tier, có đặt cảnh báo chi phí.

Giai đoạn 4: Red Hat (học song song)

Song song với AWS, em sẽ học theo lộ trình chứng chỉ **RHCSA (EX200)**. Đây là kỳ thi thực hành hoàn toàn trên hệ thống RHEL thật, hiện dựa trên RHEL 10, và các bài làm phải vẫn hoạt động đúng sau khi khởi động lại. Nội dung thi trùng khớp với nền tảng em đang có: storage và LVM, user và permission, systemd, firewalld, mạng và SELinux. Vì vậy hai hướng học sẽ bổ trợ cho nhau. Em sẽ nâng môi trường lab lên phiên bản tương thích RHEL 10, đào sâu SELinux (phần em đang yếu nhất), và học thêm Podman, công cụ container mặc định trên RHEL. Sau RHCSA, hướng tiếp theo em nhắm tới là RHCE (EX294) về tự động hóa bằng Ansible.

Em rất mong nhận được góp ý của anh/chị về lộ trình này, đặc biệt là những kỹ năng mà team đang cần, để em ưu tiên thứ tự học cho phù hợp với công việc thực tế. Em cảm ơn anh/chị đã dành thời gian lắng nghe.